The Backend Engineer's Survival Guide

Mastering, Inspecting, and Debugging Headless Distributed Systems

A Practical Manual for Developing with FastAPI, Celery, Redis, and PostgreSQL Without a Frontend

> ***Why 'Backend-First' is a Superpower:*** *In distributed systems engineering, building without a UI is standard practice. A frontend only reflects the state of the system; the actual reliability, consensus, state machine integrity, and concurrency guarantees happen strictly in the backend. This guide teaches you the exact tools, inspection techniques, and mental models senior engineers use to 'see' and control the entire distributed platform directly from your terminal and browser.*

1. The 4 Lenses of Headless Backend Engineering

When you don't have a graphical user interface, you inspect the system through four distinct operational lenses:

Lens	Component	Primary Tool	What You Are Inspecting & Verifying
1. Ingress Lens	FastAPI	Swagger UI (/docs) cURL / HTTPie	Submitting requests, triggering validation errors, receiving status codes (200, 202, 404, 409, 503).
2. Broker Lens	Redis 7	redis-cli Redis Insight	Verifying task messages enqueued, checking queue lengths, observing message consumption in real time.
3. Execution Lens	Celery Workers	Terminal logs celery inspect	Watching workers claim jobs, run tasks, apply backoff countdowns, and report success or failure.
4. State Lens	PostgreSQL 16	psql CLI DBeaver / TablePlus	Checking the authoritative single source of truth: task state transitions, version counters, attempt history.

2. Lens 1: FastAPI Swagger UI (Your Built-In Frontend)

FastAPI automatically generates an interactive graphical web interface. You do not need to wait for a custom frontend to test and interact with your backend:

• **Access Swagger UI:** Start your server with uvicorn src.main:app --reload and visit **http://localhost:8000/docs** in any browser.
• **One-Click Execution:** Click on any endpoint (e.g. GET /health/ready), click **'Try it out'**, then click **'Execute'**. You instantly see the exact HTTP response code, response headers, and formatted JSON output.
• **Live Schema Documentation:** Swagger renders the Pydantic schema for every request body, showing required vs optional fields and type constraints.
• **Alternative ReDoc View:** Visit **http://localhost:8000/redoc** for clean, publication-style documentation.

3. Lens 2: Terminal HTTP Testing Recipes

Senior backend engineers frequently test endpoints directly from the command line using curl or httpie:

```
# 1. Check Liveness Probe (Returns HTTP 200 {"status": "alive"})
curl -i http://localhost:8000/health/live

# 2. Check Readiness Probe (Checks Postgres & Redis connectivity)
curl -i http://localhost:8000/health/ready

# 3. Submit a Computational Task (Prime Factorization)
curl -i -X POST http://localhost:8000/api/v1/tasks \
  -H "Content-Type: application/json" \
  -d '{"task_type": "compute.prime_factorization", "payload": {"n": 982451653}}'

# 4. Submit a Configurable Delayed Task (Simulated Sleep)
curl -i -X POST http://localhost:8000/api/v1/tasks \
  -H "Content-Type: application/json" \
  -d '{"task_type": "delayed.sleep_and_echo", "payload": {"delay_seconds": 5}}'

# 5. Poll Task Status and Attempt History
curl -i http://localhost:8000/api/v1/tasks/<TASK_UUID>

# 6. Request Task Cancellation
curl -i -X POST http://localhost:8000/api/v1/tasks/<TASK_UUID>/cancel
```

4. Lens 3: Inspecting Authoritative State in PostgreSQL

PostgreSQL is where true state lives. When you want to verify what happened behind the scenes, you query the tables directly using the psql CLI or visual GUI clients:

```
# Connect to PostgreSQL container or local database
psql -h localhost -U postgres -d automation_db

# Inside psql prompt:
-- 1. List all tables and row counts
\dt+

-- 2. Inspect the latest submitted tasks
SELECT id, task_type, status, version, retry_count, created_at, updated_at
FROM tasks
ORDER BY created_at DESC LIMIT 5;

-- 3. Inspect execution attempts (worker identity, duration, error trace)
SELECT attempt_number, worker_id, status, duration_seconds, error_message, started_at, completed_at
FROM task_execution_attempts
WHERE task_id = '<TASK_UUID>'
ORDER BY attempt_number ASC;

-- 4. Check registered workers and last heartbeat timestamps
SELECT worker_id, hostname, status, concurrency, last_heartbeat_at
FROM workers;
```

Recommended Visual Database Clients (Free & Cross-Platform):

- **DBeaver:** Full-featured database management GUI with table inspection, query editor, and visual ER diagrams.
- **TablePlus:** Modern, lightning-fast native client with inline row editing and real-time updates.
- **VS Code Extension:** Install 'PostgreSQL' by Chris Kolkman to query tables directly inside your editor.

5. Lens 4: Inspecting Operational Messaging in Redis

Redis moves task messages between the orchestrator and Celery workers. You can watch tasks travel through Redis queues in real time:

```
# Connect to Redis CLI
redis-cli

# Inside redis-cli:
-- 1. Ping Redis server
PING                          # Returns PONG

-- 2. Inspect all active keys in the database
KEYS *

-- 3. Check how many tasks are waiting in the queue
LLEN celery                   # Returns queue depth (e.g. 0 if workers are fast)

-- 4. View task payloads waiting in queue (without removing them)
LRANGE celery 0 -1

-- 5. Monitor all Redis commands in real-time (live stream)
MONITOR                       # Shows RPUSH from FastAPI and BRPOP from Celery!
```

Visual Redis Tool: Redis Insight: Official free visual tool from Redis Ltd. Provides visual key browsers, queue stream visualizers, and memory profiling. You can run it via Docker: docker run -d -p 5540:5540 redis/redisinsight.

6. Celery Worker Inspection & Flower Web Dashboard

Celery provides built-in command-line inspection and a real-time web monitoring dashboard called **Flower**:

• **Inspect Active Workers:** celery -A src.worker.celery_app.celery_app status
• **Inspect Running Tasks:** celery -A src.worker.celery_app.celery_app inspect active
• **Flower Web Dashboard:** Install Flower via pip install flower and start it with:
celery -A src.worker.celery_app.celery_app flower --port=5555
Visit **http://localhost:5555** to view worker CPU load, task execution charts, failure rates, and retry counts graphically!

7. The 4-Step Backend Development Loop

When implementing or testing any new task type or feature, senior backend engineers use this 4-step verification loop:

```
[Step 1: Ingress]  ----->  [Step 2: Broker]  ----->  [Step 3: Worker]  ----->  [Step 4: State]
 Submit via Swagger         Watch queue depth         Watch terminal log       Query PostgreSQL
 or cURL (Get 202)          in redis-cli (LLEN)       for execution attempt    for SUCCEEDED & attempts
```

Step 1: Submit the Task via Swagger or cURL

Open http://localhost:8000/docs or run a cURL command. Submit the request. Verify you receive HTTP 202 Accepted with a JSON body containing the generated id and status QUEUED.

Step 2: Observe Operational Enqueueing in Redis

In a terminal running redis-cli monitor, verify that an RPUSH celery command was issued by the orchestrator.

Step 3: Watch Celery Worker Process Logs

In your Celery worker terminal, watch the structured logs. You will see:
[INFO] Received task: compute.prime_factorization[uuid]
[INFO] Task compute.prime_factorization[uuid] succeeded in 0.042s

Step 4: Verify Final Persistent State in PostgreSQL

Run SELECT status, version, result FROM tasks WHERE id = ''; in psql. Verify the status is SUCCEEDED, the version incremented to 2, and the output result is recorded. Query task_execution_attempts to verify duration and worker attribution.

8. Interactive Python REPL: Inspecting Code Without HTTP

You can test database queries and domain logic directly inside an interactive Python shell without running the web server:

```
$ python
>>> from src.core.database import get_sync_session
>>> from src.models import TaskRecord, WorkerRecord
>>>
>>> with get_sync_session() as session:
...     tasks = session.query(TaskRecord).all()
...     print(f"Total tasks in database: {len(tasks)}")
...     for t in tasks:
...         print(f"Task {t.id}: status={t.status}, version={t.version}")
```

9. Roadmap: When and How the Frontend Connects Later

Because we strictly adhere to clean REST principles and Pydantic schemas, when we are ready to build a frontend later (e.g. Next.js, React, or Vue), the frontend will simply be a presentation layer over these exact endpoints:
• **Task Dashboard:** Calls GET /api/v1/tasks to display a paginated table of tasks with live badges.
• **Submit Modal:** Calls POST /api/v1/tasks with user-selected parameters.
• **Worker Fleet Monitor:** Calls GET /api/v1/workers to display a live server topology diagram with green/yellow/red health lights.
• **Cancellation Button:** Calls POST /api/v1/tasks/{id}/cancel.
Until then, you have full graphical control via Swagger UI, DBeaver, and Redis Insight!